

Parte práctica del Treball de Recerca

Rediseño del enfoque: pregunta de recerca, hipótesis, metodología, plan de código, análisis y README

Repositorio: urdodev/Algebra-lineal-en-IA-i-ML · **Fecha del documento:** 8 de octubre de 2026 · **Excluye:** la memoria (TR-Memòria.pdf)

Idioma. La pregunta de recerca y las citas de hipótesis originales se mantienen en catalán, que es el idioma del TR. El resto del documento está en castellano para trabajar entre tutor y alumno. Al pasar a limpio, el texto de cada sección se puede volver a redactar en catalán.

¿Queda mucho trabajo? No: es asumible y no hace falta simplificar el núcleo. Todo el experimento son unos 1.750 ajustes de modelos pequeños (minutos en un portátil) y casi todo el código es un único bucle. El esfuerzo grande no está en programar, sino en **escribir el análisis** y la parte algebraica. Para protegerte, el plan (sección 9) separa tres niveles: **Núcleo** (imprescindible), **Recomendado** y **Extra**. Si falta tiempo se recorta de abajo arriba y la pregunta de recerca sigue quedando respondida.

Contenido

1. Resumen del cambio y por qué · 2. Clustering frente a supervisados · 3. Pregunta de recerca · 4. Hipótesis revisadas · 5. Metodología · 6. Marco algebraico · 7. Cambios en el repositorio y código · 8. Plan de análisis de resultados · 9. Plan de trabajo y limitaciones · 10. README.md propuesto · 11. Info-Dataset.md propuesto · 12. metodologia.md propuesto

1. Resumen del cambio de enfoque y por qué

	Proyecto anterior	Proyecto nuevo
Datasets	1 (Telco)	3: Telco (7.032 filas), Iris (150), Wine (178)
Modelos	Reg. logística, árbol + PCA y K-Means	KNN, reg. logística, árbol. K-Means solo como extra
Idea central	Comparar supervisados con no supervisados	Estudiar precisión, estabilidad , escalado, tamaño y repetición
Evaluación	Una partición (semilla 42, 70/30)	50 particiones estratificadas por dataset, media $\pm$ desviación típica
Métricas	Accuracy, matriz de confusión, ROC, silhouette	Accuracy (principal), balanced accuracy, baseline; extra: ARI y NMI

Por qué se cambia.

- **Comparabilidad.** Con tres modelos supervisados y la misma métrica, cada diferencia en los resultados tiene una causa identificable. En el enfoque anterior se comparaban cosas que miden magnitudes distintas (sección 2).
- **Una pregunta medible.** Variabilidad, escalado, tamaño y repetición se pueden manipular una a una y medir con números, que es lo que pide un trabajo de investigación.
- **Un solo dataset no permite hablar de tamaño ni generalizar.** Con tres datasets de tamaños y naturaleza distintos sí se pueden extraer conclusiones algo más sólidas (y se reconocen sus límites).
- **Se aprovecha lo ya hecho.** La limpieza de Telco, el baseline Dummy (73,41 %), la estructura de carpetas y el código de K-Means se reutilizan; nada del trabajo anterior se tira.

2. Clustering frente a supervisados: por qué no se puede comparar “a pelo” y cómo se hace como extra

2.1 Por qué no vale una gráfica directa

- **Distinto objetivo.** Un modelo supervisado aprende la relación entrada → etiqueta usando las etiquetas. K-Means nunca las ve: solo agrupa por cercanía.
- **Distinta métrica.** La accuracy mide aciertos frente a la realidad. El silhouette mide lo compactos y separados que están los grupos, no si son “correctos”. Poner 0,78 de accuracy junto a 0,25 de silhouette compara magnitudes distintas.
- **Las etiquetas de cluster son arbitrarias.** El “cluster 0” puede ser churn o no churn según la inicialización. Sin emparejar clusters con clases reales no existe “acierto”.
- **El número de grupos lo fija quien programa (k).** Elegir k igual al número de clases ya es darle información de las etiquetas.
- **Ventaja desigual.** El supervisado ve la respuesta correcta durante el entrenamiento; el no supervisado no.

2.2 Cómo sí se puede comparar (experimento extra E5)

- **Métricas externas.** ARI (Adjusted Rand Index) y NMI (Normalized Mutual Information) comparan los grupos con las clases reales sin depender de cómo se llamen los clusters. Son la medida principal del extra.
- **Accuracy tras emparejar.** Cada cluster se asigna a la clase mayoritaria que tiene en **train** (con el algoritmo húngaro, `scipy.optimize.linear_sum_assignment`) y esa asignación se aplica al test. Así las etiquetas solo se usan para “nombrar” los clusters, no para ajustar K-Means.
- **Mismas particiones.** K-Means se ejecuta sobre las mismas 50 particiones que los supervisados, de modo que se pueden comparar medias y desviaciones.

2.3 Limitaciones que hay que escribir en el trabajo

- La comparación sigue favoreciendo al supervisado (ve las etiquetas) y a K-Means se le da la ventaja de conocer k.
- K-Means asume grupos redondeados de tamaño parecido; si las clases no tienen esa forma (probable en Telco), puntúa mal aunque los datos sean informativos.
- El silhouette (ya calculado en tu código) se conserva como dato descriptivo, pero **no entra** en la comparación de precisión.
- PCA no es un predictor: se queda fuera de la comparación y, como mucho, se usa para dibujar los datasets en 2D.

3. Pregunta de recerca

Pregunta principal (redacción propuesta, en catalán):

Repetint l'entrenament amb diferents particions de dades, quin model supervisat (KNN, regressió logística o arbre de decisió) és més precís a l'hora de predir cadascun dels tres datasets (Telco, Iris i Wine)? Quant varia el resultat entre les diferents particions? Afecten el resultat l'escalat de les dades i la mida del dataset? Afecta d'alguna manera entrenar el model més d'una vegada sobre les mateixes dades?

Pregunta adicional (extra): *Pot un model no supervisat (K-Means) assolir resultats comparables als dels models supervisats quan s'avalua amb mètriques externes, i per què aquesta comparació no és equitativa?*

Qué se ha tocado respecto a tu versión y por qué. Se ha cerrado la redacción (“cadascun dels tres datasets”), se ha separado “varia entre particions” de “entrenar més d'una vegada” (son dos fenómenos distintos, ver hipótesis 4) y se ha añadido el extra como pregunta aparte para que su fracaso o éxito no condicione la pregunta principal.

	Subpregunta	Qué se manipula	Qué se mide	Experimento
P1	¿Qué modelo es más preciso en cada dataset?	Modelo y dataset	Accuracy media (50 particiones); balanced accuracy	E1
P2	¿Cuánto varía el resultado entre particiones?	Semilla de la partición	Desviación típica, rango e IQR de la accuracy	E1
P3	¿Afecta el escalado?	Sin escalar / MinMax	Diferencia pareada de accuracy por partición	E2
P4	¿Afecta el tamaño del dataset?	n de Telco: 150, 500, 2.000, 7.032	Media y desviación típica según n	E3
P5	¿Afecta entrenar más de una vez?	Nº de reentrenamientos (misma partición)	Nº de valores de accuracy distintos	E4
Extra	¿Se acerca K-Means a los supervisados?	Modelo (K-Means vs supervisados)	ARI, NMI, accuracy tras emparejar	E5

4. Hipótesis revisadas

Una hipótesis es una predicción que el experimento puede refutar. Se conserva tu idea original y se precisa para que sea comprobable. Fallar una hipótesis no es un problema: la parte de análisis consiste precisamente en explicar por qué.

	Tu hipótesis original	Versión revisada (comprobable)	Por qué el cambio
H1	"La pregunta 1 no es podrà respondre fins fer l'experiment."	En Iris y Wine los tres modelos superarán claramente al baseline y quedarán cerca entre sí. En Telco, la regresión logística igualará o superará a KNN, y el árbol sin podar será el peor por sobreajuste. Criterio: "más preciso" = mayor accuracy media; si la diferencia es menor que una desviación típica, es empate técnico.	No saber el resultado es correcto, pero la hipótesis debe arriesgar una predicción. Así hay algo que contrastar.
H2	El resultado depende más del tamaño que de las particiones; Iris y Wine variarán más.	La desviación típica de la accuracy entre particiones será mayor en Iris y Wine que en Telco, y disminuirá al crecer n en el submuestreo de Telco, aproximadamente en proporción a $1/\sqrt{n_{\text{test}}}$.	El tamaño y las particiones no son dos causas rivales: el tamaño del test <i>modula</i> cuánto varía el resultado entre particiones. La intuición original (más datos → más representativo) es correcta.
H3	El escalado afectará porque los valores grandes no dominarán; con 0-1 serán "más precisos".	El escalado afectará mucho a KNN, algo a la regresión logística (convergencia y regularización) y nada o casi nada al árbol.	Escalar no hace los valores "más precisos": los pone en una escala comparable. Y no afecta a todos los modelos igual (sección 6).
H4	Entrenar repetidamente afectará poco o nada porque "la memoria del modelo se reinicia".	Reentrenar sobre la misma partición dará resultados idénticos en KNN y regresión logística. El árbol solo podrá variar si no se fija random_state.	La conclusión se mantiene, pero la causa es el determinismo del algoritmo; además, fit en scikit-learn parte de cero salvo que se active warm_start.
H5 (extra)	—	K-Means quedará claramente por debajo de los supervisados en Telco (los clusters no se alinean con el churn), se acercará en Iris y mejorará en Wine con escalado.	Hace explícito qué se espera de la comparación del extra y por qué.

Orden de magnitud para H2. El error típico de una accuracy medida sobre n_{test} ejemplos es aproximadamente $\sqrt{(p(1-p)/n_{\text{test}})}$. Con test del 30 %: Iris (45 ejemplos) $\approx 3,2$ puntos si $p \approx 0,95$;

Wine (54) $\approx$ 3,0 puntos si $p \approx 0,95$; Telco (2.110) $\approx$ 0,9 puntos si $p \approx 0,80$. Son aproximaciones solo del ruido de muestreo del test; la variación real también incluye cómo cambia el entrenamiento. Sirven para saber qué desviación típica es plausible antes de ver los datos.

5. Metodología

5.1 Datasets

Dataset	Filas × variables	Clases	Origen	Tratamiento
Telco Customer Churn	7.032 × 21 (tras limpiar)	2 (Churn $\approx$ 26,6 %)	Kaggle (ya limpiado en tu repo)	Se reutiliza Telco_customers_processed.csv. Baseline Dummy: 73,41 %
Iris	150 × 4	3 (50 por clase)	sklearn.datasets.load_iris	Sin limpieza
Wine	178 × 13	3 (59/71/48)	sklearn.datasets.load_wine	Sin limpieza

Por qué estos tres. Cubren tres tamaños ($\approx$ 150, $\approx$ 180 y $\approx$ 7.000), clasificación binaria y multiclase, y variables en escalas muy distintas (Wine y Telco) frente a variables parecidas (Iris). Eso hace visible el efecto del escalado y del tamaño.

5.2 Modelos y parámetros

Modelo	Parámetros	Justificación
KNN	KNeighborsClassifier(n_neighbors=5), distancia euclídea (por defecto)	k=5 es el valor por defecto y evita añadir una “optimización de k” que desviaría la pregunta. La distancia euclídea es la fórmula del módulo de tu memoria (2.3.3.1).
Regresión logística	LogisticRegression(max_iter=1000), regularización L2 por defecto	Se sube max_iter de 200 a 1000: sin escalar, 200 iteraciones pueden no converger y entonces el “efecto del escalado” sería un efecto de convergencia, no del modelo. Se registran igualmente los avisos de convergencia.
Árbol de decisión	DecisionTreeClassifier(random_state=42), sin poda	Con random_state fijo, en E1-E3 la variación proviene solo de la partición. Sin poda mantiene la comparación con tu trabajo anterior. Variante opcional: max_depth=5.
K-Means (extra)	KMeans(n_clusters = n° de clases, n_init=10, random_state=semilla), StandardScaler	n_init=10 estabiliza la inicialización. Se mantiene el StandardScaler de tu código actual. Aquí k = n° de clases es una ventaja que se declara (2.3).

5.3 Diseño experimental

Exp.	Qué se hace	Ajustes	Responde a
E0	Caso ilustrativo (ya existe): Telco, semilla 42; matriz de confusión y ROC corregida (con probabilidades).	3	Continuidad con tu trabajo
E1	3 datasets × 3 modelos × 50 particiones (70/30 estratificadas, semillas 0-49), con MinMaxScaler.	450	P1, P2
E2	Igual que E1 pero sin escalar, sobre las mismas particiones.	450	P3
E3	Telco submuestreado de forma estratificada a n = 150, 500 y 2.000 (el de 7.032 sale de E1); 3 modelos × 50 semillas, con MinMax.	450	P4
E4	Una partición fija; 20 reentrenamientos por modelo y dataset (árbol con random_state fijo y con None).	$\approx$ 240	P5
E5 (extra)	K-Means sobre las mismas 50 particiones de cada dataset; ARI, NMI y accuracy tras emparejar.	150	Pregunta extra

5.4 Reglas de control (lo que hace válido el experimento)

- **Pipeline.** El escalador va dentro de un Pipeline, de modo que se ajusta solo con train y se aplica al test. Si se ajusta con todos los datos, el test “se cuele” en el entrenamiento (data leakage) y los resultados salen inflados.
- **Misma partición para todos los modelos.** Cada semilla genera una partición que usan los tres modelos y ambos escalados. Esto permite comparar de forma *pareada* (diferencia por partición) y es mucho más fino que comparar medias sueltas.
- **Estratificación (stratify=y).** Mantiene la proporción de clases en train y test; es clave en Telco (26,6 % de churn) y en Iris/Wine, donde un test de ~50 ejemplos podría quedar desbalanceado por azar.
- **Semillas fijas (0-49).** El experimento es reproducible y cualquiera puede repetirlo.
- **Un único factor cambia cada vez.** En E2 solo cambia el escalado; en E3 solo n; en E4 solo el n° de repeticiones.

5.5 Métricas

Métrica	Papel	Por qué
Accuracy	Principal	Es la que pide la pregunta (“más precís”) y la que ya explicas en la memoria.
Baseline Dummy (most_frequent)	Referencia	En Telco, predecir siempre “No churn” ya da 73,41 %. Sin esta referencia, un 79 % parecería mucho mejor de lo que es.
Balanced accuracy	Secundaria	Corrige el desequilibrio de clases de Telco (media de la recall por clase).
Desviación típica, rango, IQR	Variabilidad	Cuantifican “quant varia el resultat”. El boxplot las muestra a la vez.
Diferencia pareada Δ y n° de particiones donde $A > B$	Comparación	Como todos los modelos comparten partición, Δ por semilla mide mejor quién gana.
ARI, NMI, accuracy tras emparejar	Solo extra	Métricas externas para K-Means (sección 2.2).
Silhouette	Descriptiva (extra)	Mide estructura, no acierto; no entra en la comparación.

Cautela estadística. Las 50 particiones comparten buena parte de sus datos de entrenamiento, así que no son independientes. Por eso las desviaciones típicas y los porcentajes de victorias se presentan de forma descriptiva y no como pruebas de significación. Si se quiere un contraste formal (por ejemplo Wilcoxon sobre las Δ), se menciona que sus p-valores serán optimistas.

5.6 Herramientas

Python 3, pandas (datos), scikit-learn (modelos, Pipeline, métricas), scipy (emparejado de clusters en el extra) y matplotlib (boxplots y barras). Se guardan los resultados crudos en CSV (una fila por dataset, semilla, modelo y escalado) para poder rehacer cualquier gráfica sin reentrenar.

6. Marco algebraico (para la parte de análisis)

Aquí está el vínculo con el título del trabajo. Cada modelo se relaciona con una operación de álgebra lineal y eso explica por qué el escalado les afecta de forma distinta.

Elemento	Expresión	Qué explica
MinMax como transformación	$x' = D (x - m)$, con $D = \text{diag}(1/r_1, \dots, 1/r_p)$, $m = \text{vector de mínimos y } r_j = \text{máx}_j - \text{mín}_j$	Es una transformación afín (traslación + matriz diagonal). En la memoria 2.3.2 se llama “lineal”; conviene matizar que, estrictamente, la traslación la hace afín.

Elemento	Expresión	Qué explica
KNN	$d(x, y) = \ x - y\ = \sqrt{(\sum_j (x_j - y_j)^2)}$. Tras MinMax: $d' = \sqrt{(\sum_j ((x_j - y_j)/r_j)^2)}$	Sin escalar, las variables de rango grande (en Telco, TotalCharges, del orden de miles) dominan la suma y las binarias casi no cuentan. MinMax divide cada término por r_j^2 , igualando su peso.
Regresión logística	$p(y=1 x) = \sigma(w \cdot x + b)$, $\sigma(z) = 1/(1 + e^{-z})$. Frontera: hiperplano $w \cdot x + b = 0$	Es un producto escalar más una sigmoide. Si $x' = (x - m)/r$, los pesos pasan a $w' = r \odot w$. Sin regularización el modelo sería equivalente, pero la penalización L2 actúa sobre $\ w\ ^2$, así que el escalado cambia la solución; además mejora el condicionamiento y la convergencia del optimizador.
Árbol de decisión	Cada nodo pregunta $x_j \leq t$, solo sobre una coordenada	MinMax es una función creciente en cada variable: no cambia el orden de los valores, y por tanto tampoco las particiones posibles. Se esperan árboles con las mismas decisiones (los umbrales t cambian de valor, no de posición).
K-Means (extra)	Centroide = media del grupo; asignación por distancia euclídea mínima	Misma sensibilidad a la escala que KNN. Los centros se pueden ver como vectores promedio.

7. Cambios en el repositorio y en el código

7.1 Estructura propuesta

```

Data/raw/                Telco_customers_raw.csv
Data/processed/          Telco_customers_processed.csv
scr/datacleaner/         Neteja_dades_Telco.py      (rutas y index=False corregidos)
scr/supervised/
  datasets.py            carga Telco, Iris y Wine (X, y)
  train_supervised.py    E0: caso ilustrativo (ROC con probabilidades)
  experiment.py          E1, E2, E3, E4 -> outputs/results/*.csv
  plots.py              boxplots y barras a partir de los CSV
scr/unsupervised/
  train_unsupervised.py  (se conserva) + kmeans_extra.py (E5)
scr/test/
  test_train_code.py     Dummy en los 3 datasets
outputs/results/         CSV con resultados crudos
outputs/figures/         gráficas finales
Part-teorica/            TR-Memòria.pdf

```

7.2 Qué se corrige o cambia, y por qué

Archivo	Cambio	Por qué
train_supervised.py	La curva ROC se calcula con <code>predict_proba(X_test)[: , 1]</code> , no con las etiquetas predichas.	Con etiquetas 0/1 la ROC se reduce a un punto y el AUC deja de significar lo que debe.
train_supervised.py	<code>max_iter</code> de 200 → 1000; <code>stratify=y</code> ; añadir KNN.	Convergencia sin escalar; proporción de clases; pide la pregunta nueva.
Neteja_dades_Telco.py	Corregir nombres de entrada y salida, y guardar con <code>index=False</code> .	Ahora lee <code>Telco-costumers.csv</code> y escribe <code>Dades_netes_Telco_costumers.csv</code> , que no coinciden con los archivos del repo; el índice guardado obliga a borrar "Unnamed: 0".
Neteja_dades_Telco.py	No se cambia la selección de columnas, pero se documenta como limitación.	Eliminar <code>gender</code> , <code>PhoneService</code> , etc. según el churn de todo el dataset usa etiquetas antes de dividir (fuga leve de información).
metodologia.md y README.md	Reescribir (secciones 10 y 12).	Ahora describen 4 modelos y un solo dataset.
test_train_code.py	Extender el Dummy a Iris y Wine.	Da el baseline de cada dataset.

7.3 Esqueleto del experimento principal (E1 + E2)

Se ha comprobado que este esqueleto se ejecuta sobre tus datos; hay que ampliarlo con E3, E4 y el guardado en CSV.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score

MODELS = {
    "KNN": lambda: KNeighborsClassifier(n_neighbors=5),
    "LogReg": lambda: LogisticRegression(max_iter=1000),
    "Tree": lambda: DecisionTreeClassifier(random_state=42),
}
SCALERS = {"sin_escalar": None, "minmax": MinMaxScaler}

def run(name, X, y, n_splits=50):
    rows = []
    for seed in range(n_splits):
        X_tr, X_te, y_tr, y_te = train_test_split(
            X, y, train_size=0.7, stratify=y, random_state=seed)
        for m_name, make_model in MODELS.items():
            for s_name, make_scaler in SCALERS.items():
                sc = "passthrough" if make_scaler is None else make_scaler()
                pipe = Pipeline([("scaler", sc), ("model", make_model())])
                pipe.fit(X_tr, y_tr)
                pred = pipe.predict(X_te)
                rows.append(dict(dataset=name, seed=seed, model=m_name,
                                scaling=s_name,
                                acc=accuracy_score(y_te, pred),
                                bal_acc=balanced_accuracy_score(y_te, pred)))
    return pd.DataFrame(rows)
```

8. Plan de la parte de comentario y análisis de resultados

Mantiene tu estructura (contrastar hipótesis con realidad, hablar de causas, tocar el álgebra de forma superficial y concluir) y la organiza por pregunta, de modo que cada apartado tenga siempre la misma forma:

- **Resultado:** tabla (media $\pm$ desviación típica) y una gráfica.
- **Hipótesis frente a realidad:** se cumple, se cumple parcialmente o se refuta, con el número que lo demuestra.
- **Causas:** sea acertada o no la hipótesis; aquí entra el marco algebraico de la sección 6.
- **Matrices:** qué no se puede afirmar con estos datos.

Apartado	Material de resultados	Enlace con el álgebra
P1 · Precisión por modelo y dataset	Tabla de medias; boxplot por dataset; comparación con el Dummy	Frontera lineal (reg. logística) frente a umbrales (árbol) frente a distancias (KNN)
P2 · Variación entre particiones	Desviación típica, rango, IQR; comparación con el error típico teórico de la sección 4	—
P3 · Escalado	Barras con/sin escalar por modelo; Δ pareada; avisos de convergencia	MinMax como transformación afín; distancia con pesos $1/r^2$; efecto L2
P4 · Tamaño	Gráfica de la desviación típica frente a n en Telco; comparación con Iris/Wine	—
P5 · Repetición	Tabla: nº de valores distintos por modelo	Determinismo de cada algoritmo

Apartado	Material de resultados	Enlace con el álgebra
Extra · Clustering	Boxplot de ARI/NMI/accuracy emparejada junto a los supervisados	Distancia al centroide; limitaciones de la sección 2.3

Estructura propuesta para la conclusión (completa el “s'acabarà concloent el treball amb...”): (1) qué modelo gana en cada dataset y con qué margen; (2) qué tan estables son los resultados; (3) qué enseña el escalado sobre cada modelo; (4) qué pasa con el tamaño; (5) qué cambia al repetir el entrenamiento; (6) valoración del extra; (7) limitaciones y trabajo futuro (ajustar k y la profundidad del árbol, validación cruzada anidada, más datasets); (8) vuelta al título: el papel del álgebra lineal en lo observado.

9. Plan de trabajo y limitaciones

Nivel	Tarea	Si no hay tiempo
Núcleo	datasets.py; experiment.py con E1+E2 y E4; guardar CSV; boxplots y barras; análisis de P1-P3 y P5; actualizar README y metodología	No se puede recortar
Recomendado	E3 (tamaño en Telco); E0 con ROC corregida; balanced accuracy	Se recorta E3 y se reconoce que Iris/Wine/Telco difieren en más cosas que el tamaño
Extra	E5 (K-Means con ARI/NMI); árbol max_depth=5; StandardScaler; PCA en 2D	Se elimina sin perder la respuesta a la pregunta principal

Limitaciones que se deben escribir:

- Hiperparámetros fijos (k=5, árbol sin podar): el “modelo más preciso” lo es con esta configuración, no en términos absolutos.
- Iris y Wine son datasets pequeños y muy trabajados; sus resultados no se pueden generalizar a problemas reales.
- Las 50 particiones no son independientes (comparten datos de entrenamiento).
- Tamaño y naturaleza del dataset están mezclados al comparar los tres; por eso existe E3.
- La selección de columnas de Telco usó el churn de todo el dataset (fuga leve).
- En el extra, K-Means conoce k y el supervisado ve las etiquetas: la comparación no es equitativa.

10. README.md propuesto

```
# TR – Àlgebra lineal en IA i ML (part pràctica)

Part pràctica del Treball de Recerca de Catalunya.
Autor: Rayan el Bakkali · Institut: INS Vinyes Velles

## Descripció
Es comparen tres models supervisats (KNN, regressió logística i arbre de
decisió) sobre tres datasets (Telco Customer Churn, Iris i Wine). Es mesura
la precisió (accuracy), com varia entre 50 particions entrenament/test, i
l'efecte de l'escalat de dades, de la mida del dataset i de reentrenar el
model. Com a apartat extra, s'avalua K-Means amb mètriques externes i es
discuteix per què la comparació amb els supervisats no és equitativa.

## Pregunta de recerca
Repetint l'entrenament amb diferents particions de dades, quin model
supervisat és més precís a l'hora de predir cadascun dels tres datasets?
Quant varia el resultat entre particions? Afecten l'escalat i la mida del
dataset? Afecta entrenar el model més d'una vegada?

## Estructura del repositori
Data/                dades brutes (raw) i netes (processed) de Telco
scr/datacleaner/      neteja de Telco
scr/supervised/        experiment principal (E1-E4) i cas il·lustratiu (E0)
scr/unsupervised/      K-Means i PCA (apartat extra)
scr/test/              baseline amb DummyClassifier
outputs/              resultats (CSV) i gràfiques
Part-teorica/          memòria del TR
metodologia.md         metodologia detallada
Info-Dataset.md        informació dels datasets

## Models
KNN: distància euclidiana entre vectors (àlgebra: norma d'una resta).
Regressió logística: producte escalar  $w \cdot x + b$  i funció sigmoide.
Arbre de decisió: preguntes sobre una sola variable (sense àlgebra lineal).
K-Means (extra): distància al centroide.

## Mètriques
Accuracy (principal), balanced accuracy, baseline Dummy, desviació típica
entre particions. Extra: ARI, NMI i accuracy després d'aparellar clusters.

## Com reproduir
pip install pandas scikit-learn scipy matplotlib
python scr/datacleaner/Neteja_dades_Telco.py
python scr/supervised/experiment.py
python scr/supervised/plots.py

## Limitacions
Hiperparàmetres fixos; datasets petits (Iris, Wine); particions no
independents; selecció de columnes de Telco feta amb tot el dataset.

## Estat
[x] Neteja de Telco          [ ] Experiments E1-E4
[x] Baseline Dummy          [ ] Gràfiques i anàlisi
[ ] Extra E5 (opcional)     [ ] Conclusions
```

11. Info-Dataset.md propuesto

```
# Datasets utilitzats

## Telco Customer Churn (Kaggle)
https://www.kaggle.com/datasets/blastchar/telco-customer-churn
- 7.043 files originals; 7.032 després de netejar (11 files amb TotalCharges buit).
- Variable a predir: Churn (0 = No, 1 = Yes), ≈ 26,6 % de Yes.
- Conté dades brutes (nuls camuflats, columnes inútils), per això es neteja.

## Iris (scikit-learn: load_iris)
- 150 files, 4 variables numèriques, 3 classes equilibrades (50 cada una).
```

```
## Wine (scikit-learn: load_wine)
- 178 files, 13 variables numèriques, 3 classes (59 / 71 / 48).
- Variables en escales molt diferents: ideal per estudiar l'escalat.

## Per què aquests tres
Tres mides (≈150, ≈180, ≈7.000), classificació binària i multiclasse, i
variables amb escales diferents.
```

12. metodologia.md propuesto

```
# Metodologia

Datasets: Telco (net), Iris i Wine. Models: KNN (k=5), regressió logística
(max_iter=1000) i arbre de decisió (random_state=42).

Partició: 70 % entrenament / 30 % test, estratificada, amb 50 llavors (0-49).
Totes les llavors són compartides per tots els models i escalats.

Escalat: MinMaxScaler dins d'un Pipeline (s'ajusta només amb entrenament) i
versió sense escalar. Per a l'extra, StandardScaler abans de K-Means.

Experiments: E0 cas il·lustratiu (matriu de confusió i ROC amb
probabilitats); E1 50 particions amb escalat; E2 sense escalat; E3 Telco
submostrejat a 150, 500 i 2.000 files; E4 reentrenament sobre una partició
(20 repeticions); E5 (extra) K-Means amb ARI, NMI i accuracy aparellada.

Mètriques: accuracy, balanced accuracy, desviació típica, rang i IQR, Δ aparellada
entre models, baseline Dummy.

Eines: Python 3, pandas, scikit-learn, scipy, matplotlib.
```